SLGA™ Previews

Software Requirements Standard (SRS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

The Software Requirements Standard (SRS) governs how software requirements are identified, specified, assessed for completeness, accepted, changed, and traced throughout the software lifecycle. It establishes the conditions under which requirements form a legitimate and applicable basis for software development, regardless of whether they originate from humans, organizations, or AI systems.

SRS governs Software Requirements Governance.

Requested Functionality ≠ Governed Software Requirement. A software requirement must have an identifiable purpose, legitimate basis, defined applicability, and traceable governance state before it can serve as a governed development reference.

→ View Standard

Software Creation Standard (SCS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

The Software Creation Standard (SCS) governs how software is designed, generated, implemented, assembled, and transformed through identifiable, authorized, and evidence-supported creation processes. It establishes the conditions under which software creation activities and resulting artifacts remain aligned with governed requirements, legitimate authority, defined execution boundaries, and reconstructable creation evidence, regardless of whether creation is performed by humans, organizations, AI agents, or autonomous systems.

SCS governs Software Creation Governance.

Software Creation Capability ≠ Software Creation Authority. The ability to generate functional software does not establish the authority to create it. Software creation must remain connected to governed requirements, legitimate execution boundaries, identifiable artifacts, and reconstructable evidence before it can be recognized as a governed lifecycle activity.

→ View Standard

Software Dependency Standard (SDS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

The Software Dependency Standard (SDS) governs the identification, introduction, evaluation, authorization, maintenance, change, and continuity of internal and external software dependencies upon which a software system relies. It establishes the methodological conditions under which dependencies may be recognized as legitimate, suitable, compatible, evidence-supported, and governable throughout the software lifecycle. SDS distinguishes technical dependency availability from governed acceptance, dependency functionality from operational sustainability, and dependency inclusion from legitimate authority to introduce it.

SDS governs Software Dependency Governance.

Software Dependency Availability ≠ Governed Software Dependency Acceptance.

→ View Standard

Software Configuration Standard (SCFS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

The Software Configuration Standard (SCFS) governs the identification, establishment, authorization, composition, validation, change, and lifecycle continuity of software configuration states. It establishes the conditions under which configuration baselines, build processes, artifact versions, parameters, and environment-specific settings remain identifiable, internally consistent, evidence-supported, and traceable to the configuration actually established throughout the software lifecycle.

SCFS governs Software Configuration Governance.

Declared Software Configuration ≠ Governed Software Configuration State. The existence of configuration files, approved baselines, or successful builds does not establish that the intended configuration has actually been realized. Software configuration must remain identifiable, authorized, verifiable, and traceable to its effective state before it can be recognized as governed.

→ View Standard

Software Quality Standard (SQS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

The Software Quality Standard (SQS) governs the definition, assessment, verification, validation, acceptance, and lifecycle continuity of software quality. It establishes the conditions under which software quality requirements, assessment criteria, testing evidence, observed behavior, defect resolution, and acceptance decisions remain identifiable, verifiable, and traceable to the software artifact, configuration, and intended use actually evaluated.

SQS governs Software Quality Governance.

Declared Software Quality ≠ Demonstrated Software Quality. Successful tests, favorable quality metrics, or formal acceptance do not independently establish software correctness or fitness for intended use. Software quality must remain connected to defined criteria, competent evidence, observed behavior, known defects, residual limitations, and explicit acceptance conditions before it can be recognized as governed.

→ View Standard

Software Security Standard (SSS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

The Software Security Standard (SSS) governs the establishment, assessment, verification, determination, acceptance, and lifecycle continuity of software security conditions. It establishes the methodological conditions under which software security can be recognized as an identifiable, evidence-supported, and context-dependent governance state. SSS governs security requirements, protected assets, threats, attack surfaces, exposure, protective controls, security testing, vulnerabilities, remediation verification, residual risk, and security acceptance. It distinguishes declared security from demonstrated security, implemented controls from verified effectiveness, and security conformity from authorized risk acceptance.

SSS governs Software Security Governance.

Declared Software Security ≠ Demonstrated Software Security. Core Question: How can software security be defined, demonstrated, assessed, and accepted through competent evidence without mistaking implemented controls, successful scans, or closed vulnerabilities for proof of sufficient protection? Principal Output: Governed Software Security State

→ View Standard

Software Release Standard (SRLS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

The Software Release Standard (SRLS) governs the identification, preparation, readiness assessment, authorization, recording, restriction, and withdrawal of software releases throughout the software lifecycle. It establishes the methodological conditions under which an identifiable software release candidate may become an authorized release

through verified requirements alignment, applicable quality and security conditions, configuration integrity, evidence-supported readiness, and legitimate release authority. SRLS distinguishes successful software creation from release readiness, technical verification from legitimate authorization, and release approval from deployment permission. It preserves the applicability of release decisions when software artifacts, dependencies, configurations, or participating actors change.

SRLS governs Software Release Governance.

Software Release Readiness ≠ Software Release Authorization ≠ Deployment Authorization.

→ View Standard

Software Deployment Standard (SDPS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

SDPS governs how authorized software releases enter real operational environments under clearly defined authority, conditions, and control. It ensures that every deployment has an identifiable purpose, target environment, responsible actor, and verifiable operational outcome. It distinguishes permission to release software from permission to deploy it into production, where its behavior may affect users, infrastructure, and critical operations. It requires software running in production to remain identifiable and consistent with authorized versions, configurations, and deployment conditions. It governs automated pipelines and autonomous AI agents so that technical deployment capabilities never replace legitimate operational authority. It establishes deployment-state monitoring and controlled intervention when software fails, operational conditions change, or recovery becomes necessary. SDPS ensures that software is not merely deployed, but introduced, verified, controlled, and recovered under legitimate and traceable governance.

SDPS governs Software Deployment Governance Canonical Definition: SDPS governs how authorized software releases enter real operational environments under clearly defined authority, conditions, and control. It ensures that every deployment has an identifiable purpose, target environment, responsible actor, and verifiable operational outcome. It distinguishes permission to release software from permission to deploy it into production, where its behavior may affect users, infrastructure, and critical operations. It requires software running in production to remain identifiable and consistent with authorized versions, configurations, and deployment conditions. It governs automated pipelines and autonomous AI agents so that technical deployment capabilities never replace legitimate operational authority. It establishes deployment-state monitoring and controlled intervention when software fails, operational conditions change, or recovery becomes necessary. SDPS ensures that software is not merely deployed, but introduced, verified, controlled, and recovered under legitimate and traceable governance..

→ View Standard

Software Change Management Standard (SCMS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

Software Change Management Standard (SCMS) establishes how software changes are identified, justified, assessed, coordinated, directed through applicable lifecycle governance processes, verified, and formally resolved while preserving traceability between the original change requirement and the resulting software state.

SCMS governs Software Change Management Governance.

Every software change must remain governed by what it alters, why the alteration is necessary, which lifecycle mechanisms are affected, and whether the resulting state can be legitimately and verifiably established. Core Question: How can software changes remain identifiable, justified, coordinated, authorized, traceable, and verifiably resolved throughout the software lifecycle? Operational Role: SCMS governs change requests, impact assessment, coordination, lifecycle re-entry, implementation traceability, and closure without replacing specialized software lifecycle standards. Core Modules: Change Request Management (CRMM), Change Impact Assessment (CIAM), Change Coordination Control (CHCM), Lifecycle Re-entry Control (LRECM), and Change Closure Management (CCLM). Canonical Closing Statement: A software change is not governed merely because it was implemented; it is governed when its purpose, consequences, authority, lifecycle requirements, and resulting operational state can be

established through evidence. View Standard

→ View Standard

Software Retirement Standard (SRTS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

Software Retirement Standard (SRTS) is the Parent Standard within SLGA™ that governs the controlled retirement of software products, versions, components, services, dependencies, and operational capabilities. It establishes software retirement as an evidence-supported lifecycle transition rather than a simple discontinuation, deletion, or termination of support. SRTS governs retirement planning, end-of-support conditions, software migration, operational withdrawal, residual dependencies, and retirement closure. It requires the intended retirement state to be reconciled with actual software conditions, continuing dependencies, data relationships, operational responsibilities, and unresolved obligations.

SRTS governs Software Retirement Governance.

Software retirement is complete only when the intended operational withdrawal has been established and remaining dependencies, data relationships, support conditions, and obligations have been appropriately resolved, transferred, or explicitly retained under governance. Core Question: How can software be legitimately, safely, and verifiably retired when its operational use, support obligations, dependencies, data, users, authority relationships, and residual responsibilities may continue beyond the decision to discontinue it? Operational Role: Establishes the governance requirements for identifying, planning, authorizing, transitioning, withdrawing, verifying, and formally closing software retirement while preserving dependency accountability, operational continuity, and residual obligations. View Standard

→ View Standard

Software Lifecycle Control Standard (SLCS) Category: Governance & Enforcement Subcategory: Software Lifecycle Governance

Software Lifecycle Control Standard (SLCS) establishes the cross-cutting governance framework that maintains consistency, legitimacy, continuity, and verifiability of software lifecycle states, transitions, exceptions, authority relationships, and control mechanisms across the complete software lifecycle. It ensures that declared lifecycle conditions remain aligned with actual operational reality and that governance remains effective across all ten specialized Parent Standards of SLGA™.

SLCS governs Software Lifecycle Control Governance.

A software lifecycle remains governed only when its declared states, actual operational conditions, authorized transitions, responsible actors, and supporting evidence remain consistent throughout the lifecycle. Core Question: How can software remain identifiable, controllable, and governable throughout its lifecycle when its states, transitions, actors, dependencies, operational conditions, authority relationships, and governance requirements continuously change? Operational Role: Establishes continuous lifecycle state consistency, transition control, exception governance, control transfer and continuity, and evidence-supported assessment of lifecycle control effectiveness. Canonical Closing Statement: Software lifecycle governance is not simply the governance of individual stages. It is the preservation of legitimate control across every transition between them. View Standard

→ View Standard